

Repository-Grounded Audit Supplement for SpecFlow

Anonymous Authors

1 Scope and Evidence Boundary

This supplement converts the repository audit in `paper\_draft\_specflow.md` into LaTeX form. It is grounded in the local repository at `C:/Users/Administrator/Desktop/specflow`. The current checkout is a ZIP-extracted directory rather than a git clone. The local repository contains source code, README, configs, tests, experiment launch specs, and bundled paper-reported baseline metrics, but it does not contain `data/`, `outputs/`, trained checkpoints, logs, `results.csv`, or `agg\_results.csv`.

Therefore, this supplement distinguishes three categories:

1. facts directly supported by source code or configuration files;
2. plausible interpretations of the method that require experimental validation;
3. claims that cannot be made until run artifacts are available.

2 Repository-Grounded Method Report

2.1 Problem Solved

SpecFlow targets single-cell perturbation response prediction. Given unperturbed control cells and a perturbation condition represented as one or more targeted genes, the model predicts the post-perturbation gene-expression distribution. The scientifically central target is the perturbation effect relative to control expression, not merely expression reconstruction.

2.2 Inputs and Outputs

The data pipeline and model use:

- `ctrl\_expr`: control-cell expression with shape (B, G) ;
- `pert\_expr`: perturbed-cell expression with shape (B, G) during training;
- `pert\_mask`: binary gene-aligned perturbation mask with shape (B, G) ;
- `condition`: perturbation condition string;
- `spectral\_embedding`: either a tensor of spectral coordinates or a mapping with `go` and `coexp` eigenvectors.

The model predicts a per-gene velocity vector. During inference, an Euler sampler integrates the learned velocity field from a noisy control state to a predicted perturbed expression state.

2.3 Training Flow

The control-anchored flow objective in `src/specflow/flow/flow\_matching.py` uses:

$$x_0 = x_c + \sigma\epsilon, \quad x_t = (1 - t)x_0 + tx_1, \quad u_t = x_1 - x_0.$$

The model minimizes mean-squared error between predicted and target velocity. If `flow.ot\_coupling` is enabled, the implementation reorders perturbed samples within each condition using a Hungarian assignment before computing targets.

The trainer additionally supports:

- `delta\_corr\_weight`: condition-grouped perturbation-delta correlation loss;
- `mmd\_weight`: optional MMD loss;
- EMA weights, AMP, cosine schedule, and warmup.

2.4 Core Modules

Table 1: Core repository modules used by SpecFlow.

Area	Files
Data	<code>src/specflow/data/dataset.py</code> , <code>benchmark.py</code> , <code>preprocessing.py</code>
Graphs	<code>graph/go_graph.py</code> , <code>coexp_graph.py</code> , <code>spectral_embedding.py</code> , <code>perturbation_aware.py</code>
Model	<code>model/specflow.py</code> , <code>spectral_fusion.py</code> , <code>gene_encoder.py</code> , <code>velocity_field.py</code> , <code>contextual_propagation.py</code>
Flow	<code>flow/flow_matching.py</code> , <code>ode_solver.py</code> , <code>mmd_loss.py</code>
Training	<code>training/trainer.py</code> , <code>training/ema.py</code> , <code>experiment.py</code>
Evaluation	<code>evaluation/scdfm_protocol.py</code> , <code>evaluator.py</code> , <code>results.py</code> , <code>metrics.py</code>

3 Supported Method Claims

The following claims are currently supported as implementation facts:

1. SpecFlow implements control-anchored flow matching for perturbation prediction.
2. SpecFlow implements GO and coexpression graph construction.
3. SpecFlow computes spectral gene embeddings and fuses dual graph features.
4. SpecFlow supports condition-wise OT coupling.
5. SpecFlow supports a delta-correlation auxiliary loss.
6. SpecFlow can write scDFM-style `pred.h5ad` and `real.h5ad` evaluation files.

The following claims require experiments:

1. graph spectra improve performance;
2. OT coupling improves performance;
3. delta-correlation training improves perturbation-effect metrics;
4. SpecFlow outperforms any baseline;
5. the contextual-local propagation extension is superior to the default spectral propagation path.

4 Potential Narrative Mismatches

README versus default configs. The README’s architecture section emphasizes `graph\_pool` and `contextual\_local` propagation. However, `configs/norman.yaml` and `configs/norman\_holdout.yaml` do not enable those options; they inherit the default `legacy` perturbation encoder and `spectral` propagation variant. **Need confirmation:** The final paper must choose the actual main method variant and make README, configs, and manuscript consistent.

Evaluation alignment. The code supports scDFM-compatible evaluation, but the local checkout lacks the external data, split files, and run artifacts needed to verify that all experiments are actually aligned.

Mechanistic visualizations. The experiment plan discusses UMAPs, top-DE gene plots, and graph propagation heatmaps, but no figure outputs exist locally.

5 Experiment Plan

The minimal evidence package required for a submission-ready paper is:

1. Norman additive full model, fold 0–4;
2. Control and Additive baselines, fold 0–4;
3. Norman holdout full model;
4. holdout Single/Double result split;
5. core ablations: `graph\_none`, `no\_spectral\_propagation`, `no\_ot\_coupling`, `no\_delta\_corr`, `no\_control\_anchor`;
6. extra metrics: L2, Pearson Delta-hat, Pearson Delta-hat20;
7. mechanistic visualizations;
8. runtime and resource reporting.

6 Current Code/Test Audit

`python -m pytest -q` currently reports 57 passed and 2 failed in this Windows environment. The failures are:

1. `tests/test\_dataset\_protocol\_configs.py::test\_holdout\_configs\_use\_generated\_folded\_p`
UTF-8 YAML is read with the locale default encoding, causing a `GBK UnicodeDecodeError` on Chinese comments.
2. `tests/test\_launch\_experiments.py::test\_shell\_command\_prefers\_current\_checkout\_source`
the test expects a generated shell command to include `PYTHONPATH=<tmp>/src`, but the observed command string does not include that exact fragment.

These failures do not affect the LaTeX conversion, but they are relevant to repository reproducibility.

7 Submission-Readiness TODO List

- **TODO:** Add required datasets and split files.
- **TODO:** Run main additive and holdout experiments.
- **TODO:** Run baselines under the same split, gene set, and evaluation protocol.
- **TODO:** Run component ablations.
- **TODO:** Fill all paper tables from verified run artifacts.
- **TODO:** Fix or document the two current pytest failures.
- **TODO:** Resolve the main-method version ambiguity.
- **TODO:** Add citations and replace placeholder related-work text.
- **TODO:** Remove or explicitly retain all TODO markers before submission.